

Exercise 5: Connect & Extend

Session: 5 — MCP Servers & External Tools **Duration:** 30 minutes **Project:** Same BookStore API.

Goal: Build a custom SQLite MCP server that exposes the bookstore database to Claude. Then create a security-auditor subagent, and compare AI responses with and without each extension loaded.

Stuck on a question, or short on time? Say *"just tell me"* to Claude and it will skip ahead to a direct answer.

Tasks

1. Build the SQLite MCP Server (8 min)

The bookstore uses a SQLite database (`store.db`). Without an MCP, Claude doesn't know anything about the database and has to guess column names. With MCP, it can access information about the database and knows to query the schema and run real SQL. The server lives at `mcp-sqlite/main.go` and exposes two tools:

- `get_table_definitions` — returns `CREATE TABLE` statements and column info
- `execute_query` — executes a read-only `SELECT` query and returns JSON rows

Step 1 — Make sure the bookstore database exists. From the project root, start the bookstore server once (it seeds the database on the first run):

```
cd bookstore-ts && bun install && bun run start &
# wait ~2 seconds, then stop it — we just need store.db to exist
kill %1
cd ..
```

Verify the `store.db` file exists:

```
ls -lh bookstore-ts/store.db
```

Step 2 — Compile the MCP server to a binary. Claude Code launches the server as a subprocess and talks to it over stdin/stdout, so it needs an executable:

```
cd mcp-sqlite && go build -o ../mcp-sqlite-server . && cd ..
```

This produces `mcp-sqlite-server` in the project root. Confirm it compiled:

```
./mcp-sqlite-server --help
```

You should see the `-db` flag printed. If so, the server is working.

Step 3 — Register the server with Claude Code (project scope). Use the absolute path so Claude Code can find the binary regardless of working directory, and point it at `bookstore-ts/store.db` :

```
claude mcp add \
  --transport stdio \
  --scope project \
```

```
sqlite-bookstore \  
-- $(realpath mcp-sqlite-server) -db $(realpath bookstore-ts/store.db)
```

Step 4 — Verify the server is registered:

```
claude mcp list
```

You should see `sqlite-bookstore` in the list. Inside an active Claude Code session, type `/mcp` to confirm it shows as connected and lists both tools: `get_table_definitions` and `execute_query`.

Done when: `claude mcp list` shows `sqlite-bookstore`, and `/mcp` shows it connected with both tools listed.

2. Compare AI Responses: Without vs With MCP (5 min)

Round A — without MCP (disable it temporarily):

```
claude mcp remove sqlite-bookstore
```

Open a fresh Claude Code session. You are about to ask it this:

"How many books are in the bookstore database? Which author has the most books? Write a SQL query that returns all books with their author name and average rating, sorted by rating descending."

Before you send it, bet on which of the three answers will come out wrong: the book count, the top author, or the SQL query. Claude has no database access yet, so at least one of them is likely to be wrong.

Now ask it.

Write down Claude's response. Notice:

- Does it hesitate or add caveats about not knowing the schema?
- Are the column names correct (`author_id` , `review_text` , `rating`)?
- Does it join the right tables?

Round B — with MCP (re-add it):

```
claude mcp add \  
--transport stdio \  
--scope project \  
sqlite-bookstore \  
-- $(realpath mcp-sqlite-server) -db $(realpath bookstore-ts/store.db)
```

Open a **new** Claude Code session (so MCP connects on startup) and ask the **exact same question**.

Observe the difference:

- Claude now calls `get_table_definitions` first — watch for the tool call in the output
- It then calls `execute_query` with a real query
- Column names are exact, joins are correct, results are real data

Ask a follow-up that would be impossible without live data:

"Which book has the highest average rating? Show me its title, author, and the top 3 review texts."

Without MCP this is just a guess. With MCP it is a fact.

Done when: you have written responses for both rounds.

3. Create the Security-Auditor Subagent (5 min)

A subagent runs in its own isolated context window with its own tools and model. You will create one that specializes in OWASP security audits. It uses a cheaper model (Haiku) and only gets read access, so it can never modify code.

Step 1 — Create the agents directory:

```
mkdir -p bookstore-ts/.claude/agents
```

Step 2 — Create `bookstore-ts/.claude/agents/security-auditor.md` with this content:

```
---
name: security-auditor
description: >
  Audits TypeScript source code for OWASP Top 10 security vulnerabilities.
  Invoke this agent whenever the user asks for a security review,
  vulnerability check, or when new handlers or store functions are added.
tools: Read, Grep, Glob
model: claude-haiku-4-5-20251001
effort: xhigh
color: red
---
```

You are a security engineer specializing in TypeScript / Node-style web applications and the OWASP Top 10. Your job is to find real vulnerabilities – not theoretical risks.

Scope

Audit only the files you are given. Do not modify any file.

Process

1. Run ``Glob`` with ``src/**/*.ts`` to find all source files
2. For each handler file in ``src/handler/``, read it fully
3. For each store file in ``src/store/``, read it fully
4. Check for the following vulnerabilities:

****A01 – Broken Access Control****

- Are there authorization checks on any endpoint?
- Can an unauthenticated user call DELETE or POST endpoints?

****A03 – Injection****

- Are SQL queries built with template strings or concatenation instead of parameter binding?
- Are query parameters sanitized before use?

****A05 – Security Misconfiguration****

- Are error messages returned verbatim to HTTP clients?
- Does the server expose stack traces or internal paths?

****A07 – Identification and Authentication Failures****

- Is there any authentication middleware at all?

****A09 – Security Logging and Monitoring Failures****

- Are failed requests or suspicious inputs logged?

Output format

Write a report with this structure:

Security Audit Report

****Files reviewed****: list every file you read

For each finding:

****[SEVERITY] OWASP Category – Short title****

- File: `path/to/file.ts`, line N
- Description: what the vulnerability is
- Evidence: paste the relevant code snippet
- Recommendation: one concrete fix

Severity levels: CRITICAL, HIGH, MEDIUM, LOW, INFO

End with a ****Summary**** table: | Severity | Count |

Step 3 — Test whether the subagent auto-triggers. Ask Claude directly, without naming `security-auditor`:

"Do a security audit of the bookstore API."

Watch the tool calls in the output. Claude will delegate to `security-auditor` automatically because the description matches. The audit runs in a separate context — your main conversation stays clean.

Done when: `bookstore-ts/.claude/agents/security-auditor.md` exists, and you have a security-audit report produced by the subagent.

Closing round (10 min)

Write down the answers to these. The trainer may ask students to answer out loud.

1. **MCP schema awareness:** did Claude call `get_table_definitions` before every query, or only once? What does that tell you about how Claude reuses tool results within a session?
2. **Subagent delegation:** did Claude delegate automatically, or did you have to trigger it explicitly? What would you change in the `description:` field to make auto-delegation more reliable?
3. **Real-world applications:** which MCP servers would save your team the most time, and what would you need to check before connecting one?
4. **Cost vs capability:** the security auditor uses Haiku to save cost. Name one task in your own workflow that a cheaper model could handle. Say why it would not lose quality.

Then start `/verify-exercise 5`. It checks your MCP registration and your subagent's file, trigger, and report. It grades by observable state, not by prose. Bring its report to the closing round if it finishes in time.

Done when: you have a written answer for all four, and `/verify-exercise 5` is running.